

田永铭 221900180 第一次作业：说明：为了尽量节省篇幅，部分证明和图表放入了附录部分，附录仅仅作作为补充，证明作业是我做的，不占篇幅，老师可以不看。

一、数学基础部分

Problem 1.1 (a) 计算投影: $\mathbf{x}_\perp = \frac{\mathbf{x} \cdot \mathbf{y}}{\mathbf{y} \cdot \mathbf{y}} \mathbf{y} = \frac{2\sqrt{3}}{4}(1, \sqrt{3})^T = \left(\frac{\sqrt{3}}{2}, \frac{3}{2}\right)^T$. (b) $\mathbf{x} - \mathbf{x}_\perp = \left(\frac{\sqrt{3}}{2}, -\frac{1}{2}\right)^T$, $\mathbf{y} \cdot (\mathbf{x} - \mathbf{x}_\perp) = \left(\frac{\sqrt{3}}{2}, -\frac{1}{2}\right)^T \cdot (1, \sqrt{3})^T = 0$, 因此 $\mathbf{y} \perp (\mathbf{x} - \mathbf{x}_\perp)$. (c) 如图 1. (d) 由提示 $\|\mathbf{x} - \mathbf{x}_\perp\|^2 + \|\mathbf{x}_\perp - \lambda \mathbf{y}\|^2 = \|\mathbf{x} - \lambda \mathbf{y}\|^2$, 结合 $\|\mathbf{x}_\perp - \lambda \mathbf{y}\|^2 \geq 0$, 得 $\|\mathbf{x} - \mathbf{x}_\perp\| \leq \|\mathbf{x} - \lambda \mathbf{y}\|$.

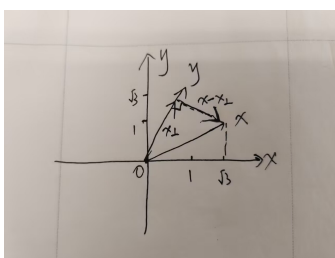


图 1: Problem 1.1-(c)

Problem 1.2 (a) X 为正定矩阵 $\Leftrightarrow x > 0$. (b) $\det(X) = 1 \cdot 1 \cdot 3 \cdot 4 \cdot x = 72$, 解得 $x = 6$.

Problem 1.3 (a) $p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right)$.

(b) $\ln p(\mathbf{x}) = -\frac{d}{2} \ln(2\pi) - \frac{1}{2} \ln |\Sigma| - \frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)$.

(c) 由 $\frac{\partial}{\partial \mu}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu) = -2\Sigma^{-1}(\mathbf{x} - \mu)$ (从 matrix cookbook Eq.(81) 变形而来), 得 $\frac{\partial \ln p(\mathbf{x})}{\partial \mu} = \Sigma^{-1}(\mathbf{x} - \mu)$.

(d) 由 $\frac{\partial}{\partial \Sigma^{-1}} \ln |\Sigma| = \frac{\partial}{\partial \Sigma^{-1}} (-\ln |\Sigma^{-1}|) = -\Sigma$ (Eq.(49)) 及 $\frac{\partial}{\partial \Sigma^{-1}} (\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu) = -(\mathbf{x} - \mu)(\mathbf{x} - \mu)^T$ (Eq.(70)), 得 $\frac{\partial \ln p(\mathbf{x})}{\partial \Sigma^{-1}} = \frac{1}{2}\Sigma - \frac{1}{2}(\mathbf{x} - \mu)(\mathbf{x} - \mu)^T$.

二、评估

Problem 2.1 查准率 (Precision): $\text{Precision} = \frac{TP}{TP+FP} = \frac{20}{20+30} = 0.4$; 查全率 (Recall)

: $\text{Recall} = \text{TPR} = 0.2$; **F1 值**: $F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times 0.4 \times 0.2}{0.4 + 0.2} = \frac{4}{15} \approx 0.2667$;

准确率 (Accuracy): $\text{Accuracy} = \frac{TP+TN}{P+N} = \frac{20+70}{100+100} = 0.45$; 错误率 (Error Rate):

$\text{Error Rate} = 1 - \text{Accuracy} = 1 - 0.45 = 0.55$

Problem 2.2 (a)

下标	类别标记	得分	查准率	查全率	AUC-PR	AP
0			1.0000	0.0000	—	—
1	1	1.0	1.0000	0.2000	0.2000	0.2000
2	2	0.9	0.5000	0.2000	0.0000	0.0000
3	1	0.8	0.6667	0.4000	0.1167	0.1333
4	1	0.7	0.7500	0.6000	0.1417	0.1500
5	2	0.6	0.6000	0.6000	0.0000	0.0000
6	1	0.5	0.6667	0.8000	0.1267	0.1333
7	2	0.4	0.5714	0.8000	0.0000	0.0000
8	2	0.3	0.5000	0.8000	0.0000	0.0000
9	1	0.2	0.5556	1.0000	0.1056	0.1111
10	2	0.1	0.5000	1.0000	0.0000	0.0000
总和					0.6907	0.7277

(b) AUC-PR 总和: 0.6907; AP 总和: 0.7277 AUC-PR 和 AP 的值应该相似 (确实相似), 因为它们都是对 PR 曲线的总结, 他们只是一个估计矩形, 一个估计梯形, 这两者的关系会随着图形单调性动态变化, 没有绝对大小, 最后总值相似。

(c) AUC-PR 为 0.6794, AP 为 0.7167。(利用下面 (d) 中代码运行得到)

(d) 首先编写纯计算 AUC-PR 和 AP 代码, 核心部分见附录 4.1.1。然后利用原数据调用 AUC-PR 和 AP 库函数计算, 验证我的计算正确, 核心部分见附录 4.1.2。运行结果见附录 4.1.3, 都是 AUC-PR 为 0.6906, AP 为 0.7278, 证明我的答案正确。

Problem 2.3 (a) $p(x) = \frac{1}{2}\mathcal{N}(-1, 0.25) + \frac{1}{2}\mathcal{N}(1, 0.25)$, 其中 $\mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$ 。

(b) 期望代价为: $E_{(x,y)}[c_{y,f(x)}] = \sum_x p(x) \left(\sum_{y \neq f(x)} p(y|x) \right)$ 即: $E_{(x,y)}[c_{y,f(x)}] = \sum_x p(x) \cdot (1 - p(f(x)|x))$, 至此显然等价; 对于 $C > 2$ 仍然成立, 因为代价矩阵中的对角线元素还是为零, 且最大化条件概率总是最小化误分类的代价。

(c)

$$f(x) = \begin{cases} 1, & p(x|y=1) > p(x|y=2) \text{ (计算得 } x < 0) \\ 2, & \text{否则} \end{cases}$$

(d) 解方程 $10 \cdot e^{-2(x+1)^2} = e^{-2(x-1)^2}$, 得 $x = \frac{\ln 10}{8}$,

$$f(x) = \begin{cases} 1, & P(x|y=2) < 10 \cdot P(x|y=1) \text{ (计算得 } x < 0.2878) \text{ (准确答案是 } \frac{\ln 10}{8}) \\ 2, & \text{否则} \end{cases}$$

三、FLD

Problem 3.1 (a) 令 A 是一个 $m \times n$ 的矩阵, 其列秩为 r , 即矩阵 A 的列空间的维度为 r 。令 $c_1, c_2, \dots, c_r$ 为 A 的列空间的一组基, 构成 $m \times r$ 的矩阵 $C = [c_1, c_2, \dots, c_r]$ 。矩阵 A 的每一列可以表示为 C 中的列向量的线性组合, 因此存在一个 $r \times n$ 的矩阵 R , 使得 $A = CR$ 。由于 $A = CR$, 矩阵 A 的每个行向量是 R 的行向量的线性组合, 因此 A 的行空间被包含于 R 的行空间中。由于 R 只有 r 行, 矩阵 R 的行秩最多为 r , 即 $\text{rank}(A) \leq \text{rank}(R) \leq r$ 。

同理, 可以交换“行”与“列”来证明 A 的列秩也不超过行秩。

- (b) 矩阵 A 的秩是其行空间或列空间的维数, 显然, 行空间的维数不超过行数 m , 列空间的维数不超过列数 n , 因此 $\text{rank}(A) \leq \min(m, n)$ 。
- (c) 考虑矩阵 $X + Y$ 的列空间。它是 X 和 Y 的列空间的并集, 而并集的维数不超过各个空间维数之和。因此, 矩阵 $X + Y$ 的秩不超过 X 和 Y 的秩之和, 即 $\text{rank}(X + Y) \leq \text{rank}(X) + \text{rank}(Y)$ 。
- (d) 矩阵 XY 的秩不超过 X 和 Y 的秩, 因为矩阵乘法导致的映射不会增加秩。即 $\text{rank}(XY) \leq \min(\text{rank}(X), \text{rank}(Y))$ 。另一种证明利用 $\text{Col}\{XY\} \subseteq \text{Col}\{X\}$ and $\text{Row}\{XY\} \subseteq \text{Row}\{Y\}$, 分别得到矩阵 XY 的秩小于 X 和 Y 。
- (e) XX^T 列空间由 X 生成, 所以 $\text{rank}(X) \leq \text{rank}(XX^T)$, 由 (d) 中结论, $\text{rank}(XX^T) \leq \text{rank}(X)$, 所以, $\text{rank}(XX^T) = \text{rank}(X)$ 。所以前一个等式成立。同理, 可得最终结论。
- (f) x 为 0 则秩为 0, 反之为 1; 综合利用本题目 (b) 和 (d) 即可得到这个结论。矩阵的秩等于其列空间的维数, 而列空间的维数与特征值有关。每个非零特征值对应一个线性独立的特征向量, 故特征值为零的部分不会贡献到矩阵的秩中。所以后一个问题得证。

Problem 3.2 (a) 不可逆。当 $N < D$ 时, 样本数小于特征数, 因此 S_W 是一个 $D \times D$ 的矩阵, 且其秩最多为 N 。具体推导如下: 不可逆。当 $N < D$ 时, 样本数小于特征数, 因此 S_W 是一个 $D \times D$ 的矩阵, 且其秩最多为 N 。具体推导如下:

由习题 3.1, 结合 $S_W = \sum_{y_i=k} S_k$, 可得 $\text{rank}(S_W) \leq \sum_{y_i=k} \text{rank}(S_k)$ 。又因为 $S_k = \sum_{y_i=k} (x_i - \mu_k)(x_i - \mu_k)^T$ 所以有 $\text{rank}(S_k) = \text{rank}\left(\sum_{y_i=k} (x_i - \mu_k)(x_i - \mu_k)^T\right) \leq \sum_{y_i=k} \text{rank}((x_i - \mu_k)(x_i - \mu_k)^T) = \sum_{y_i=k} \text{rank}((x_i - \mu_k)^T(x_i - \mu_k)) = \sum_{y_i=k} 1$ 。最终得到 $\text{rank}(S_W) \leq \sum_{y_i=k} \text{rank}(S_k) \leq \sum_{y_i=k} 1 = N < D$ 。

- (b) 设 S_B 为类间散度矩阵, S_W 为类内散度矩阵, S_B 的秩最大为 $C - 1$ (这是由于他的定义式里面有一个线性约束, 知道了前 $C-1$ 个 μ_i 后, 最后一个 μ 可以有前面的来线性表示), 所以, 广义特征值问题 $S_B w = \lambda S_W w$ 最多有 $C - 1$ 个非零特征值, 因此最多可以获得 $C - 1$ 个广义特征向量。

- (c) 当样本数大于（远大于）特征数，由于样本采得足够充分，所以不同类型的样本就多，导致 S_W 通常是一个高秩矩阵。而 S_W 的秩只要达到 D ，它就是满秩的，因此在这种情况下 S_W 很可能是可逆的。由 (a) 可知 $\text{rank}(S_w) \leq \sum_i \text{rank}(S_i) = \sum_i \sum_{C_{ij} \in C_i} 1 = N$ ，若 $N > D$ ，则有可能使得 $\text{rank}(S_w) = D \leq N$ 成立，则 S_w 可逆。若 $N \gg D$ ，则这种可能性更大，所以 S_w 可逆的可能性也更大。
- (d) 利用算法条件得 $X^T S_B X = Q^T G^{-1} S_B G^{-T} Q = Q^T C Q$ 。由于 C 已经对角化，所以 $X^T S_B X$ 是对角矩阵。对于 S_W ，我们有 $X^T S_W X = Q^T G^{-1} G G^T G^{-T} Q = I$ 。因为 S_W 可逆所以广义特征值问题等价于 $S_W^{-1} S_B \omega = \lambda \omega$ 即 $S_W^{-1} S_B$ 的特征值问题。因为 $X^{-1} S_W^{-1} S_B X = Q^{-1} G^T G^{-T} G^{-1} S_B G^{-T} Q = Q^{-1} G^{-1} S_B G^{-T} Q = X^T S_B X$ (Q 是正交阵)，根据特征值分解的方法， $S_W^{-1} S_B$ 和 S_B 相似，广义特征值应位于 $X^T S_B X$ 的对角线上，广义特征向量位于 X 的列中。

在 S_W -内积下：定义 $\langle x, y \rangle_{S_W} = x^T S_W y$ 。由 $X^T S_W X = I$ ，对任意两列 x_i, x_j ：

$$x_i^T S_W x_j = \delta_{ij} \quad (\delta_{ij} \text{ 为 Kronecker delta}).$$

因此， x_i 在 S_W -内积下正交且具有单位范数。

在标准欧氏内积下：

$$x_i^T x_j = (G^{-T} q_i)^T (G^{-T} q_j) = q_i^T G^{-1} G^{-T} q_j = q_i^T S_W^{-1} q_j.$$

由于 S_W^{-1} 一般不是单位矩阵，除非 $S_W = I$ ，否则 x_i 在标准内积下不一定正交，也不保证单位范数。综上：根据这种方式计算得到的广义特征向量在 S_W -内积下有单位范数且正交，在标准内积下不是。

Problem 3.3 (c) 我分别利用 PCA (eigenface) 和 FLD 在 ORL 数据集上进行处理，对人脸图像进行重建，参考的代码就是 opencv 官方文档[官方 Opencv 文档](#)。其中核心部分代码，eigenface 是调用的 “createEigenFaceRecognizer()”，FLD 是调用的 “createFisherFaceRecognizer()”，基本代码我已经理解，限于篇幅不在此展示，点击以上链接即可查看。运行结果分别如图 2 和图 3。从特征图来看，eigenfaces 算法生成的特征图是彩色的，有色调对比，突出了面部的关键区域。而 fisherfaces 算法的特征图主要呈现蓝紫色调，更具有条纹细节。从重建效果看，eigenfaces 重建的图像较为清晰，特征辨识度较高，可能的原因如下：在 ORL 这样的小样本数据集上，小样本问题 ($N < D$) 导致类内散布矩阵不可逆，影响效果。ORL 数据集的主要变化趋势可能由光照、表情变化主导，而不是类别信息，这使得 PCA 更有优势。

- (d) opencv 上的源码是在 AT&T Facedatabase 数据集上运行的，并且是 c++，结论是大约 300 个特征脸能够使得重构出来图像与原始图像基本无差别。我遵循本题，在 ORL 数据集上实验，观察选择特征脸数目来看多少才能达到要求。代码和 (c) 题保持一致，观察图 2，可以发现大概是 235 个特征脸已经能使得重构的效果比较好。

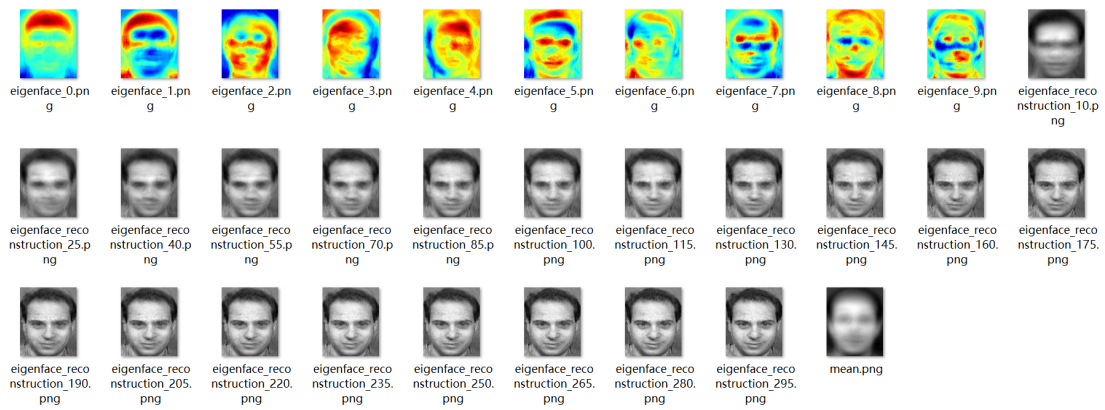


图 2: eigenface



图 3: fisherface

四、附录

4.1 代码和实验补充

4.1.1 代码 1

```

1 auc_pr = np.sum((recall[1:] - recall[:-1]) * (precision[1:] + precision[
    :-1]) / 2) # AUC-PR
2 ap = np.sum((recall[1:] - recall[:-1]) * precision[1:]) # AP

```

4.1.2 代码 2

```

1     def compute_auc_pr_ap(y_true, scores):
2         precision, recall, _ = precision_recall_curve(y_true, scores, pos_label
               =1)
3         auc_pr = auc(recall, precision) # 计算 AUC-PR
4         ap = abs(np.sum(np.diff(recall) * precision[:-1])) # 计算 AP
5         return auc_pr, ap

```

4.1.3 运行 1

```

...
● PS C:\Users\86181\Desktop\UsefulTools\download> python .\hhh.py
AUC-PR: 0.6906
AP: 0.7278
● PS C:\Users\86181\Desktop\UsefulTools\download> python .\xxx.py
AUC-PR: 0.6906
AP: 0.7278
-

```